

Pointer and Memory

Song Liu (song.liu@bristol.ac.uk)

GA 18, Fry Building,

Microsoft Teams (search "song liu").

Previously

- What is Array?
 - How to
 - create an array? `int a[4];` or `int a[]={1,2,3};`
 - access an array? `a[0] = 1;`
 - How an array is stored in the computer memory?
 - consecutive blocks of memory
- How to pass an array as an input argument of a function?
 - Passing by reference
 - Not, passing by value

Homework 5.4, Row Major Matrix

- Print a row major matrix A stored in an array.

```
int A[6] = {1,2,3,4,5,6};
for(int i = 0; i < 3; i++){
    for(int j = 0; j < 2; j++){
        // what should I put here?
    }
    printf("\n");
}
```

Today's Agenda

- **What is a pointer?**
- How to declare and initialize a pointer?
- What are "address of" and "dereference" operators?
- Array and Pointer

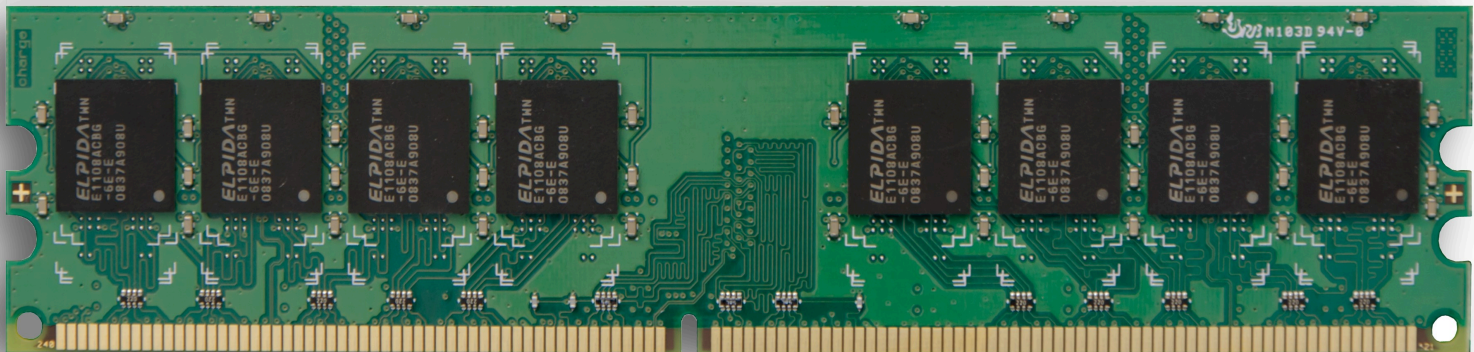
Pointer and Memory

C is different from many other programming languages, it gives you the ability to directly read/write memory that is allocated to your program.

- Make good use of this feature, your code can run much faster than programming in other languages.
- However, the "raw pointer" is not widely used in modern day programming, but it is still a very important educational tool about memory access in programming language.
 - Modern C++ uses **smart pointer**, which is basically a raw pointer with a nice wrapping paper.

Physical Memory and Virtual Memory

- Modern computers use **Random Access Memory (RAM)** to temporarily store information being used by the CPU. RAM is called the "Physical Memory" of a computer.
 - It stores machine code of programs and their data in small "cells" that are made of **MOSFET**.

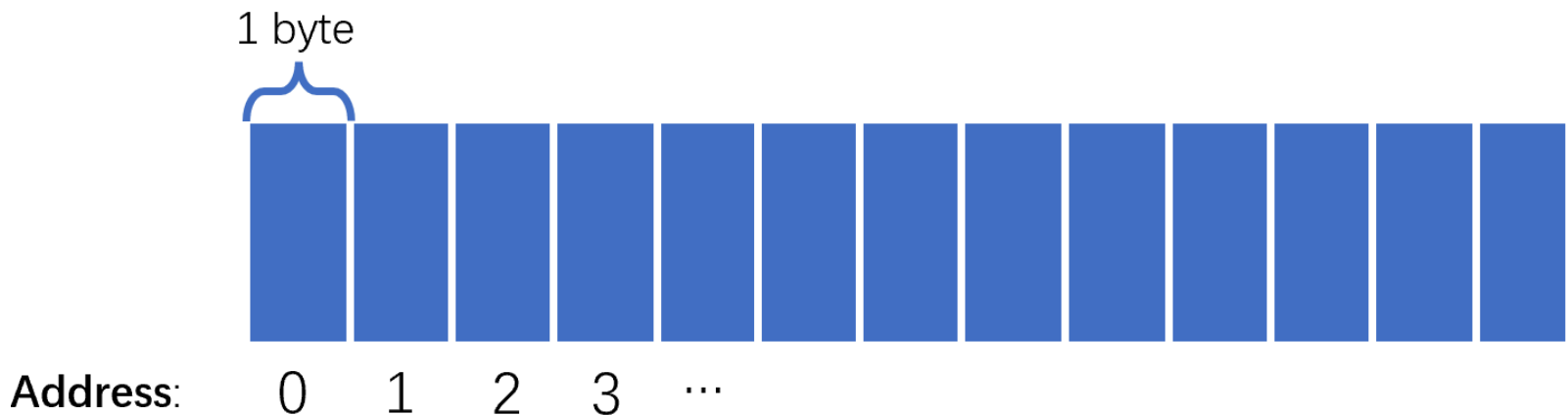


Physical Memory and Virtual Memory

- Physical memory is a precious computational resource thus should be carefully rationed and managed. Poor management of physical memory would lead to errors and severe performance degradation over time.
- Luckily, OS manages physical memory for you so you do not have to.
 - Physical memory is not visible to your program for security reasons anyway.
- Your program can only see "**Virtual Memory**".

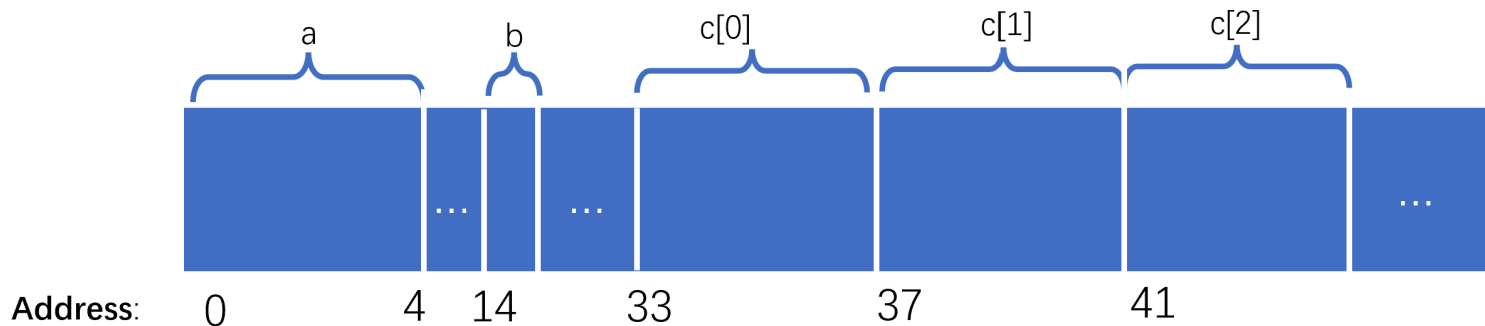
Physical Memory and Virtual Memory

- **Virtual Memory** is an **abstraction** of the physical memory, made available to your program by the OS.
- You can think of it as a huge library shelf with many small slots next to each other. Each slot is a smallest memory unit, usually a **byte** (8 binary bits).
- You can refer to a byte using its index (usually zero-based). The index is called **the address** of the byte.



Multi-byte Variables

- However, we know some of the data types in C occupies more than one byte. For example, `int` occupies 4 bytes.
- These variables will occupy consecutive bytes in virtual memory.
- Elements in an array will also be stored consecutively in the virtual memory.
- Example: `int a; char b; int c[3];`



Pointer

- Pointer is a variable that **stores the address of another variable in the virtual** memory.
- Using a pointer we can access the content stored in that memory location.
 - Like index card to a book in the library.

Today's Agenda

- What is a pointer?
- **How to declare and initialize a pointer?**
- What are "address of" and "dereference" operators?
- Array and Pointer

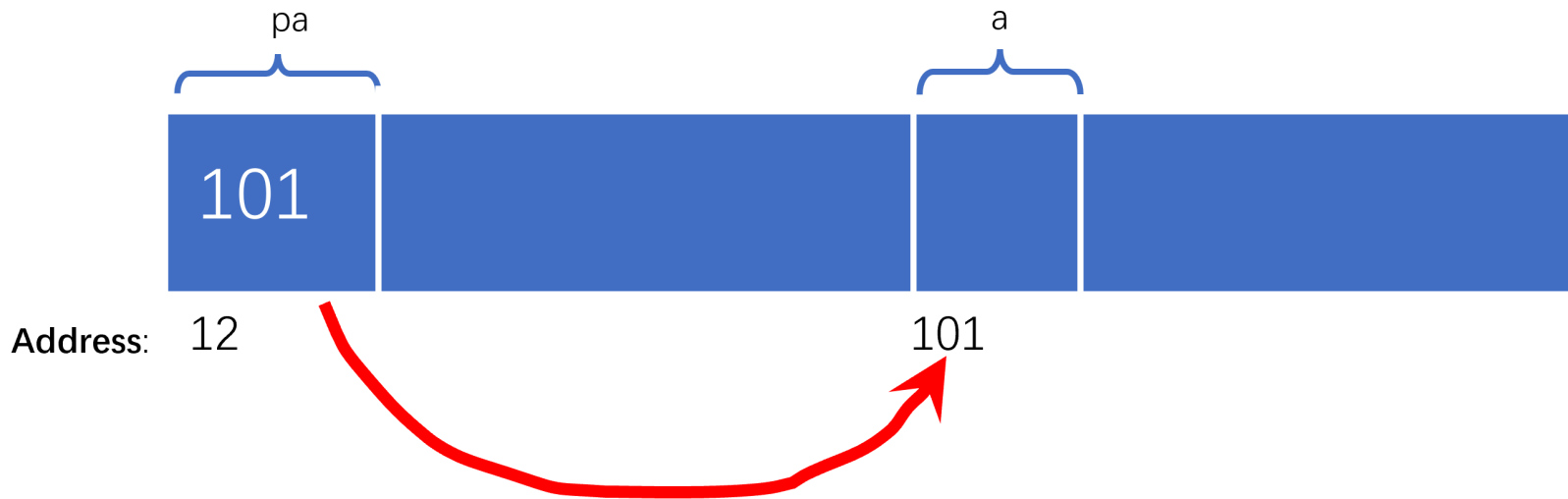
Declare and Initialize Pointer

- A pointer itself is a variable in C, thus should be declared before use.
 - Syntax: `data_type *pointer_name;`
 - `int *pa;` declares a `int` pointer `pa`.
 - `double *pb;` declares a `double` pointer `pb`.
 - `int *pa = &a;` initialize `pa` with the address of `a`.
- A pointer can be given any valid variable name.
- I recommend you use `p` prefix when naming pointers (e.g., `pa`, `pb`) to distinguish them from ordinary variables.

Memory Diagram of Pointer

```
int a = 0;  
int *pa = &a;
```

- `pa` stores the address of `int` variable `a`.
- `pa` itself is also a variable and is stored in the memory.



Pointer

- Without any initialization or assignment, the pointer points to a random memory location.

```
#include <stdio.h>
void main(){
    int *pa; //BAD!
    printf("I point to %p.\n", pa);
    // displays "I point to <some random memory space>".
}
```

- Trying to access memory in such random location will result in unpredictable behaviors!!!
 - Most time Runtime error!
- Good habit: Decalare a point when you know what variable it should points to!

& Operator

- `&` is the "address of" operator. `&a` gives you the virtual memory address of `a`.

```
#include <stdio.h>
void main(){
    int a = 0;
    int *pa = &a;
    printf("The address of a is %p.\n", pa);
    printf("The address of a is %p.\n", &a);
    // displays "The address of a
    // is <some memory address>" twice.
}
```

- Use `%p` to print out a pointer.
- Don't confuse it with logic AND!

* Operator

- * is the opposite of &, called dereference operator.
- * takes the value from a certain memory address.

```
#include <stdio.h>
void main(){
    //initialize the pointer to be the address of a.
    int a = 1;
    int *pa = &a;
    int b = *pa;
    printf("The value of a is %d.\n", b);
    // displays "The value of a is 1."
}
```


Today's Agenda

- What is a pointer?
- How to declare and initialize a pointer?
- **Array and Pointer**

Pointer and Array

In fact, in C, **array name** is a **pointer pointing** to the first element of the array!!

```
#include <stdio.h>
void main(){
    int a[] = {2,3,4};
    int *pa = &a[0];
    printf("memory of a[0] is %p\n", a);
    printf("memory of a[0] is %p\n", pa);
    //prints out
    // <The memory address of the first elem of a>
    // <The memory address of the first elem of a>
}
```

Pointer and Array

You can use a **pointer pointing to the first element** to access the elements in the array!

```
#include <stdio.h>
void main(){
    int a[] = {2,3,4};
    int *pa = &a[0];
    printf("The value of a[0] is %d\n", pa[0]);
    printf("The value of a[1] is %d\n", pa[1]);
    printf("The value of a[2] is %d\n", pa[2]);
    //prints out
    // The value of a[0] is 2.
    // The value of a[1] is 3.
    // The value of a[2] is 4.
}
```

- `a` and `pa` are interchangeable!

Pass by Reference, Revisited

- Do you remember we mentioned that when arrays are passed as input arguments of a function, they are passed by reference rather than by value?
- When passing an array as an input argument, **the array name represents a pointer to the first element to the array so it is actually this pointer get passed to the function, rather than the array elements themselves.**
 - `swap(a)` , where `a` is the array name, AND a pointer to its first element.
 - This is why, arrays are passed by reference, not by value.

Conclusions

- A pointer is a variable stores the address of another variable.
- A pointer is declared as `datatype *pointername; .`
- `&` "address of" operator takes the address from a variable.
- `*` "dereference" operators takes the content from an address.
- You can use a pointer to access an array.
 - The array name and the pointer pointing to the first element of the array is interchangeable.

Homework 6.1

1. Download the lab file and put them into your labpack.
2. Read `operators.c` and pay attention to the usage of `&` and `*`.
3. Complete the TODO tasks specified in the file.

Homework 6.2

1. Read `pointers_and_arrays_2.c` and run the code.
2. Explain why the code prints out such an outcome.
3. Complete the TODO tasks specified in the file.

Homework 6.3 (Submit)

1. Open `swap.c` .
2. Write a function `swap` , that takes two `int` **pointer** inputs.
3. `swap` function swaps the contents of variables which `pa` and `pb` point to.